SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – MCQs

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – MCQs
Weightage:	10% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	**Beffy A Shanika**	**Reg. No.:**	**192511387**
Section / Batch:	**2025**	**Date:**	**12/08/2026**

Instructions to Students

- Answer all questions carefully. Each question carries 2 marks. Total: 100 marks.
- Analyze each question and select the correct answer.
- Apply concepts correctly to solve problem-based MCQs.
- Manage your time efficiently to complete all 50 questions.

Question Type	Focus Area	Questions
MCQs	Analyze and apply Divide and Conquer Algorithms for Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication.	Q1 -Q50

SECTION A – MCQs

Code of Conduct

I Beffy A Shanika(192511387) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: beffy

RUBRICS FOR CALCULATION

Online Quiz / MCQ Test					
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Conceptual Understanding	30%	Demonstrates thorough understanding; answers all questions accurately	Shows good understanding with few errors	Partial understanding; several incorrect responses	Lacks understanding; majority of answers incorrect
Accuracy of Answers	25%	All answers are correct	Most answers are correct with minor mistakes	Some correct answers; frequent errors	Mostly incorrect answers
Application of Knowledge	20%	Correctly applies concepts to problem-based questions	Applies concepts with minor errors	Limited ability to apply concepts	Unable to apply concepts
Time Management	15%	Completes quiz well within time efficiently	Completes on time with minor delays	Requires extra time or rushes through questions	Unable to complete within given time
Question Interpretation	10%	Interprets all questions correctly and responds appropriately	Misinterprets a few questions	Often misinterprets questions	Unable to understand questions

Mark Evaluations:

No. of MCQ Questions	Correct Answers	Wrong/Not Answers	Total Marks (100)
50			

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Student 6 **Beffy A Shanika** 192511387 50 / 50 submitted

Q1. Number of parts array divided in Merge Sort

Merge Sort divides the input array into smaller parts before sorting and merging them. Into how many parts is the array divided at each step?

✓ **Correct** **2.00 / 2.00**

- **A** 3 parts
- **B** 2 parts ← Student
- **C** n parts
- **D** log n parts

Q2. Strassen's algorithm multiplication reduction (2)

Strassen's algorithm optimizes matrix multiplication by reducing the number of multiplication operations. How many multiplications are reduced compared to the standard method?

✓ **Correct** **2.00 / 2.00**

- **A** 6 to 5
- **B** 8 to 7 ← Student
- **C** 9 to 7
- **D** 8 to 6

Q3. Type of arrays merged in Merge Sort

Merge Sort combines smaller subarrays after dividing the original array recursively. What type of arrays are merged during this process?

✓ **Correct** **2.00 / 2.00**

- **A** Unsorted arrays
- **B** Sorted subarrays ← Student
- **C** Random elements
- **D** Single elements only

Q4. Name of Merge Sort's combining step

Merge Sort combines sorted halves to produce a fully sorted array. What is this combining step called?

✓ **Correct** **2.00 / 2.00**

- **A** Partition
- **B** Merge ← Student
- **C** Divide
- **D** Swap

Q5. Paradigm followed by Merge Sort

> Merge Sort is a recursive algorithm that divides the problem into smaller subproblems and solves them independently. Which paradigm does it follow?

✓ **Correct** **2.00 / 2.00**

- **A** Iterative
- **B** Divide and Conquer ← Student
- **C** Greedy
- **D** Dynamic Programming

Q6. Reason Quick Sort is called in-place

> Quick Sort is considered an in-place algorithm because it minimizes additional memory usage. Why is it called in-place?

✓ **Correct** **2.00 / 2.00**

- **A** No recursion
- **B** Constant extra space ← Student
- **C** Uses merging
- **D** Uses DP

Q7. Quick Sort worst-case pivot condition

> Quick Sort performance depends on how well the pivot divides the array. When does Quick Sort exhibit worst-case behavior?

✓ **Correct** **2.00 / 2.00**

- **A** Pivot is median
- **B** Pivot divides equally
- **C** Pivot is smallest or largest ← Student
- **D** Input is random

Q8. Reason Quick Sort is unstable

> Quick Sort is not a stable sorting algorithm due to its partitioning method. What is the main reason?

✓ **Correct** **2.00 / 2.00**

- **A** Pivot selection
- **B** In-place swaps ← Student
- **C** Recursion depth
- **D** Randomization

Q9. Quick Sort average time complexity

> Quick Sort average-case performance is efficient due to balanced partitions in most cases. What is its average time complexity?

✓ **Correct** **2.00 / 2.00**

- **A** O(n)
- **B** O(n log n) ← Student
- **C** O(log n)
- **D** O(n)

Q10. Binary Search Tree traversal complexity

Binary Search tree traversal visits all nodes sequentially. Analyze the time complexity of traversal operations.

✓ **Correct** **2.00 / 2.00**

- **A** O(n) ← Student
- **B** O(log n)
- **C** O(n log n)
- **D** O(n)

Q11. Time complexity of partitioning in Quick Sort

Quick Sort performs partitioning at each recursive step. What is the time complexity of the partition operation?

✓ **Correct** **2.00 / 2.00**

- **A** O(1)
- **B** O(log n)
- **C** O(n) ← Student
- **D** O(n)

Q12. Technique used by Strassen's algorithm

Strassen's algorithm applies a recursive strategy to multiply matrices efficiently. Which technique does it use?

✓ **Correct** **2.00 / 2.00**

- **A** Greedy
- **B** Divide and Conquer ← Student
- **C** Backtracking
- **D** Brute Force

Q13. Merge Sort recurrence relation

Merge Sort follows a divide-and-conquer strategy by splitting arrays and merging sorted halves. Identify the correct recurrence relation.

✓ **Correct** **2.00 / 2.00**

- **A** T(n)=2T(n/2)+O(n) ← Student
- **B** T(n)=T(n-1)+O(1)
- **C** T(n)=T(n/2)+O(1)
- **D** T(n)=2T(n/2)+O(1)

Q14. Condition where Binary Search fails

Binary Search requires a specific condition to work correctly. When does Binary Search fail?

✓ **Correct** **2.00 / 2.00**

- **A** Small dataset
- **B** Unsorted data ← Student
- **C** Large dataset
- **D** Numeric data

Q15. Strassen's algorithm efficiency condition

Strassen's algorithm performs better than naive multiplication only under certain conditions. When is it most efficient?

✓ Correct 2.00 / 2.00

- **A** Large input size ← Student
- **B** Small input size
- **C** n = 2
- **D** n = 3

Q16. Property ensuring Merge Sort stability

Merge Sort is considered stable because it preserves the relative order of elements. What property ensures this stability?

✓ Correct 2.00 / 2.00

- **A** Preserves order during merging ← Student
- **B** Uses pivot selection
- **C** Uses recursion
- **D** Uses partitioning

Q17. Binary Search mid index formula

Binary Search computes the middle index during each step. Which formula is commonly used to find the mid value?

✓ Correct 2.00 / 2.00

- **A** (low + high) / 2 ← Student
- **B** (low + high) / 3
- **C** (low + high) / 4
- **D** (low + high) / n

Q18. Worst-case comparisons in Binary Search

Binary Search reduces the search space logarithmically with each comparison. What is the worst-case number of comparisons?

✓ Correct 2.00 / 2.00

- **A** n
- **B** log n ← Student
- **C** n log n
- **D** n

Q19. Quick Sort worst case with poor pivot

In Quick Sort, if the pivot selected is always the smallest element, what is the worst-case time complexity?

✓ Correct 2.00 / 2.00

- **A** O(n) ← Student
- **B** O(n log n)
- **C** O(log n)
- **D** O(n)

Q20. Binary Search best-case complexity

Binary Search efficiently finds an element by repeatedly dividing the search space. What is its best-case time complexity?

✓ **Correct** **2.00 / 2.00**

- **A** O(n)
- **B** O(log n)
- **C** O(1) ← Student
- **D** O(n log n)

Q21. Quick Sort best performance condition

Quick Sort achieves best performance when partitions are balanced. Under which condition does this occur?

✓ **Correct** **2.00 / 2.00**

- **A** Pivot divides array equally ← Student
- **B** Pivot is smallest
- **C** Pivot is largest
- **D** Pivot is random

Q22. Binary Search recurrence relation

Binary Search uses divide-and-conquer by halving the search space repeatedly. Identify its recurrence relation.

✓ **Correct** **2.00 / 2.00**

- **A** T(n)=T(n/2)+O(1) ← Student
- **B** T(n)=T(n-1)+O(1)
- **C** T(n)=2T(n/2)+O(n)
- **D** T(n)=T(n/2)+O(n)

Q23. Binary Search complexity for unsuccessful search

Binary Search may fail to find an element even after several comparisons. What is its complexity in an unsuccessful search?

✓ **Correct** **2.00 / 2.00**

- **A** O(1)
- **B** O(log n) ← Student
- **C** O(n)
- **D** O(n log n)

Q24. Time complexity of recursive divide-and-conquer algorithm

Consider a recurrence relation representing a divide-and-conquer algorithm with increasing recursive calls. Analyze how the complexity grows with input size. What is the time complexity?

✗ **Wrong** **0.00 / 2.00**

- **A** O(n log n) ← Student
- **B** O(n)
- **C** O(n^log n) ✓ **Correct**
- **D** O(log n)

Q25. Hybrid sort switching to Insertion Sort

A common hybrid approach switches to Insertion Sort for small subarrays. Why is this optimization used?

✓ Correct 2.00 / 2.00

- **A** Reduce recursion depth
- **B** Improve performance on small inputs ← Student
- **C** Avoid pivot selection
- **D** Reduce space complexity

Q26. Binary Search comparisons for n=1024

For an array of size 1024, determine the maximum number of comparisons required in the worst case.

✗ Wrong 0.00 / 2.00

- **A** 9
- **B** 10 ✓ Correct
- **C** 11 ← Student
- **D** 12

Q27. Strassen's algorithm complexity

Strassen's algorithm improves matrix multiplication by reducing the number of multiplications compared to the conventional method. What is its time complexity?

✓ Correct 2.00 / 2.00

- **A** O(n)
- **B** O(n.81) ← Student
- **C** O(n log n)
- **D** O(n)

Q28. Merge Sort time complexity in all cases

Merge Sort maintains consistent performance regardless of input order. What is its time complexity in all cases?

✓ Correct 2.00 / 2.00

- **A** O(n)
- **B** O(n log n) ← Student
- **C** O(log n)
- **D** O(n)

Q29. Binary Search comparisons for n=1000

For 1000 elements, determine the worst-case number of comparisons.

✓ Correct 2.00 / 2.00

- **A** 9
- **B** 10 ← Student
- **C** 11
- **D** 12

Q30. Pivot position after partitioning

> Quick Sort uses a partitioning technique where a pivot element is selected and placed in its correct position. What happens to the pivot after partitioning?

✓ **Correct** **2.00 / 2.00**

- **A** Placed randomly
- **B** Moves to correct sorted position ← Student
- **C** Moves to first index
- **D** Moves to last index

Q31. Stable sorting algorithm

> Merge Sort and Quick Sort differ in stability when handling duplicate elements. Identify the stable algorithm.

✓ **Correct** **2.00 / 2.00**

- **A** Merge Sort ← Student
- **B** Quick Sort
- **C** Both
- **D** Neither

Q32. Main advantage of Quick Sort

> Quick Sort is often preferred in practical scenarios due to its in-place nature. What is the main advantage of Quick Sort?

✓ **Correct** **2.00 / 2.00**

- **A** Always faster
- **B** Uses less auxiliary space ← Student
- **C** Stable sorting
- **D** No recursion

Q33. Strassen's algorithm reduced operation

> Strassen's algorithm is efficient because it reduces the number of a specific operation. Which operation is reduced compared to naive multiplication?

✓ **Correct** **2.00 / 2.00**

- **A** Additions
- **B** Multiplications ← Student
- **C** Memory usage
- **D** Recursion depth

Q34. Condition causing Quick Sort inefficiency

> Quick Sort performance depends heavily on pivot selection strategy. When does Quick Sort become inefficient?

✓ **Correct** **2.00 / 2.00**

- **A** Random input
- **B** Poor pivot selection ← Student
- **C** Small input
- **D** Balanced partition

Q35. Quick Sort on sorted array with first pivot

If the input array is already sorted and the first element is chosen as pivot, what case is produced?

✓ Correct 2.00 / 2.00

- **A** Best case
- **B** Worst case ← Student
- **C** Average case
- **D** None

Q36. Strassen's multiplication reduction

Compared to the standard method, how many multiplications does Strassen's algorithm reduce?

✓ Correct 2.00 / 2.00

- **A** 8 to 7 ← Student
- **B** 7 to 6
- **C** 9 to 7
- **D** 8 to 6

Q37. Binary Search precondition

Binary Search repeatedly divides the search space but works only under a specific condition. Identify the required condition.

✓ Correct 2.00 / 2.00

- **A** Randomized data
- **B** Sorted data ← Student
- **C** Balanced data
- **D** Partitioned data

Q38. Structure unsuitable for Binary Search

Identify the data structure where Binary Search is not suitable to apply efficiently.

✓ Correct 2.00 / 2.00

- **A** Sorted arrays
- **B** Linked lists ← Student
- **C** Balanced trees
- **D** Hash tables

Q39. Number of parts data divided in Binary Search

Binary Search works by dividing the dataset into smaller portions. Into how many parts is the data divided?

✓ Correct 2.00 / 2.00

- **A** Two halves ← Student
- **B** Three parts
- **C** Unequal parts
- **D** Random parts

Q40. Merge Sort auxiliary space complexity

Merge Sort requires additional memory to merge sorted subarrays. Analyze its auxiliary space complexity.

✓ Correct 2.00 / 2.00

- **A** O(1)
- **B** O(log n)
- **C** O(n) ← Student
- **D** O(n)

Q41. Merge Sort base case for recursion

Merge Sort continues dividing the array until it reaches its smallest unit. When does this division stop?

✓ Correct 2.00 / 2.00

- **A** Size becomes 2
- **B** Size becomes 1 ← Student
- **C** Size becomes n/2
- **D** Array is sorted

Q42. Average recursion depth of Quick Sort

Quick Sort creates recursive calls based on partitioning of the array. What is the average recursion depth?

✓ Correct 2.00 / 2.00

- **A** n
- **B** log n ← Student
- **C** n log n
- **D** 1

Q43. Reason Merge Sort is not in-place

Merge Sort requires extra memory to store temporary arrays during merging. Why is it not in-place?

✓ Correct 2.00 / 2.00

- **A** Uses recursion
- **B** Needs extra memory ← Student
- **C** Uses comparisons
- **D** Uses pivot

Q44. Reason Merge Sort preferred for external sorting

Merge Sort is often used in external sorting where data cannot fit into memory. Why is Merge Sort preferred?

✓ Correct 2.00 / 2.00

- **A** Faster pivot
- **B** Stability and sequential access ← Student
- **C** Less recursion
- **D** Less memory

Q45. Condition where Binary Search is most effective

Binary Search is widely used for efficient searching in large datasets. When is Binary Search most effective?

✓ Correct 2.00 / 2.00

- **A** Sorted data ← Student
- **B** Unsorted data
- **C** Large data
- **D** Duplicate data

Q46. Divide and Conquer steps

Identify the correct sequence of steps involved in the Divide and Conquer approach.

✓ Correct 2.00 / 2.00

- **A** Break ? Solve ? Combine ← Student
- **B** Iterate ? Optimize ? Return
- **C** Choose ? Compare ? Select
- **D** Store ? Reuse ? Update

Q47. Element used for comparison in Binary Search

Binary Search compares the target value with a specific element in the dataset at each step. Which element is used for comparison?

✓ Correct 2.00 / 2.00

- **A** First element
- **B** Last element
- **C** Middle element ← Student
- **D** Random element

Q48. Merge Sort preferred over Quick Sort for linked lists

Merge Sort is preferred over Quick Sort in certain scenarios such as linked lists. What is the primary reason?

✓ Correct 2.00 / 2.00

- **A** Stability and sequential access ← Student
- **B** Faster pivot selection
- **C** Less recursion
- **D** Lower memory usage

Q49. Merge Sort recursion tree height

Merge Sort divides the array recursively until base cases are reached. Analyze the height of its recursion tree.

✓ Correct 2.00 / 2.00

- **A** log n ← Student
- **B** n
- **C** n log n
- **D** n

Q50. Operation minimized by Strassen's algorithm

> Strassen's algorithm achieves improved performance by reducing costly operations. Which operation is minimized?

✓ **Correct** **2.00 / 2.00**

- **A** Addition
- **B** Multiplication ← Student
- **C** Memory
- **D** Division